Equipo 16	
Nombre: Lesly Adamaris Cotlame Nieto Christopher Bethancourt Contreras Francisco Mora Ruiz	**Matrícula**: 2226958 2226948 2226912
Nombre del curso: Componentes de lógica de negocios.	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 5	**Actividad**: Actividad 5 · Integración, Pruebas y Calidad
Fecha: 27/10/2026	

Bibliografia (Formato APA)

- Astral Software. (s.f.). *Ruff* [Documentación]. https://docs.astral.sh/ruff/
- Batchelder, N. (s.f.). *Coverage.py* [Documentación]. https://coverage.readthedocs.io/
- Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley.
- Componentes de Lógica de Negocios. (2026). *Laboratorio guiado Unidad 5: Integración, pruebas y calidad* [Material del curso]. Facultad de Ingeniería Mecánica y Eléctrica, UANL.
- Cotlame Nieto, L. A., Bethancourt Contreras, C., & Mora Ruiz, F. (2026). *Reporte BRD corregido, historias de usuario y glosario de Eventos Nova* [Documentos no publicados]. FIME, UANL.
- GitHub. (s.f.). *Workflow syntax*. GitHub Docs. https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax
- Meszaros, G. (2007). *xUnit test patterns: Refactoring test code*. Addison-Wesley.
- pytest-dev. (s.f.). *How to use skip and xfail to deal with tests that cannot succeed*. pytest documentation. https://docs.pytest.org/en/stable/how-to/skipping.html
- pytest-dev. (s.f.). *pytest-cov* [Repositorio de código]. GitHub. https://github.com/pytest-dev/pytest-cov
- Python Software Foundation. (s.f.). *unittest.mock — mock object library*. Python 3 documentation. https://docs.python.org/3/library/unittest.mock.html
- SQLAlchemy. (s.f.). *Transactions and connection management*. SQLAlchemy 2.0 Documentation. https://docs.sqlalchemy.org/en/20/orm/session_transaction.html

Objetivo:

Demostrar, con pruebas automatizadas y evidencias reproducibles, que las reglas de negocio se comportan como se espera: casos de éxito, error y límite; TDD con el ciclo Red–Green–Refactor; persistencia con commit y rollback; servicios externos simulados con mocks; políticas de autorización; cobertura mínima del 80 %; revisión estática con Ruff y un pipeline de GitHub Actions. La actividad tiene dos alcances separados: el proyecto didáctico de pedidos del laboratorio y la adaptación de las pruebas a Eventos Nova, el negocio oficial de nuestro PIA.

Procedimiento:

REPORTE DE LA ACTIVIDAD 5

Unidad 5 · Integración, Pruebas y Calidad · Laboratorio base y Eventos Nova

Introducción

En las unidades anteriores convertimos las reglas de Eventos Nova en un modelo de dominio y en una aplicación en Python. En esta unidad la pregunta es otra: cómo demostrar que lo programado funciona. Para practicar trabajamos primero con el proyecto base del profesor, un sistema pequeño de pedidos, y después aplicamos lo mismo al código real de Eventos Nova.

Todo lo que se reporta aquí proviene de ejecuciones guardadas en la carpeta `evidencias/` del entregable. Las pruebas se ejecutaron el 27/10/2026 en un entorno Linux con Python 3.13.18; para Eventos Nova se usó además una base PostgreSQL 16 local y efímera. Lo que depende de GitHub (las ejecuciones reales del pipeline) quedó preparado y se marca como pendiente.

1. Procedimiento y desarrollo

1.1 Análisis del proyecto base y arquitectura

El ZIP del laboratorio contiene las instrucciones, tres CSV de datos ficticios (`casos_de_prueba.csv`, `pedidos_prueba.csv`, `clientes_prueba.csv`), una base SQLite didáctica, un script para regenerarla y un segundo ZIP con el proyecto ejecutable. Solo este último tiene código: `src/` con cuatro módulos y `tests/` con 13 pruebas. Las reglas reales del código son: `Order.confirm()` rechaza totales menores o iguales a cero y clientes con deuda, aplica 15 % de descuento VIP y deja el pedido en `CONFIRMED`; `OrderService.checkout()` confirma y cobra mediante una pasarela, y lanza `RuntimeError("payment rejected")` si el cobro no se aprueba; `SqlOrderRepository` guarda en la tabla `orders`, y `can_cancel()` aplica la política de cancelación.

Antes de modificar nada ejecutamos la suite original: 13 de 13 pruebas aprobadas y 98 % de cobertura (faltaba la rama del administrador en `security.py`). La misma revisión mostró tres problemas del entorno de calidad que se recogen en la Tabla 1, junto con el resto de los hallazgos iniciales.

Tabla 1. Hallazgos iniciales del análisis

Categoría	Hallazgo
Requisitos	Matriz de al menos 12 casos (5 éxito, 4 error, 3 límite); TDD de doble confirmación; persistencia con id 2 y total 250; mocks de pago aprobado, rechazado y timeout; matriz de autorización; cobertura ≥ 80 %; CI verde → rojo → verde; tres reglas del PIA con nueve casos como mínimo.
Funcionalidad existente	Descuento VIP, validación de total y deuda, cobro con pasarela, repositorio SQLAlchemy y política de cancelación. El código permitía confirmar dos veces (caso CP08).
Pruebas existentes	13: dominio (5), repositorio (2), servicio (2) y seguridad (4). Las de seguridad usaban la hora real.
Pruebas faltantes	VIP/regular de 200, límites de total y deuda, doble confirmación, id 2/total 250, unicidad, rollback parcial, timeout, administrador sobre pedido ajeno y propietario a las 24 h exactas.
Problemas de calidad	(1) `pytest` sin `python -m`, como lo ejecuta el workflow original, falla con `ModuleNotFoundError: src`. (2) Ruff 0.16.9, que es lo que instala `ruff>=0.6`, reporta 7 errores de orden de imports. (3) Las pruebas de repositorio dejaban conexiones SQLite abiertas (`ResourceWarning`).
Riesgos técnicos	`checkout()` confirma antes de saber si el pago se aprobó; `checkout()` no recibe la deuda del cliente; los importes usan `float`; `confirm()` no define qué hacer con estados distintos de NEW y CONFIRMED (por ejemplo, SHIPPED), así que no se inventó una regla para ese caso.
Limitaciones	Sin cuenta de GitHub vinculada en la sesión; sin acceso a PyPI (las dependencias se instalaron desde los repositorios oficiales en GitHub y desde los paquetes del entorno del equipo); no se recibió el reporte de la Actividad 4.
Diferencias entre documentos	La guía de la actividad y la documentación aprobada de Eventos Nova difieren en varias reglas (sección 1.10, Tabla 5).

Los tres problemas de calidad se corrigieron sin tocar la lógica de negocio: `pythonpath = ["."]` en `pyproject.toml`, `ruff check --fix` (solo reordena imports) y fixtures que cierran la sesión y liberan el motor. Para que no vuelvan a aparecer conexiones abiertas, `pyproject.toml` convierte `ResourceWarning` en error. El trabajo quedó registrado en Git con un commit por paso (Figura 7).

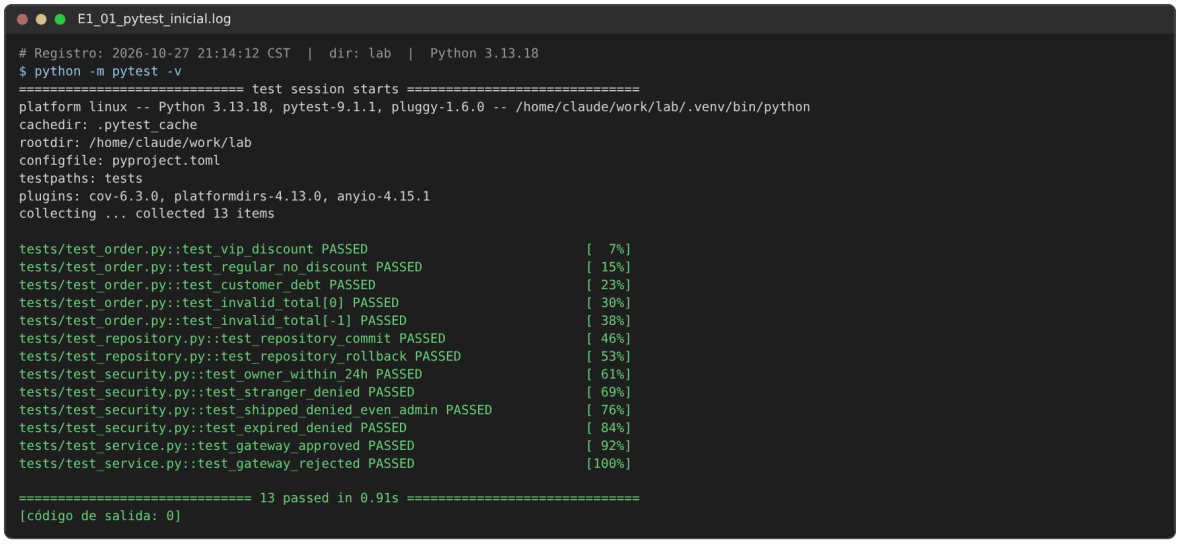


Figura 1. Ejecución inicial de la suite original (E1): 13 pruebas aprobadas antes de cualquier cambio.

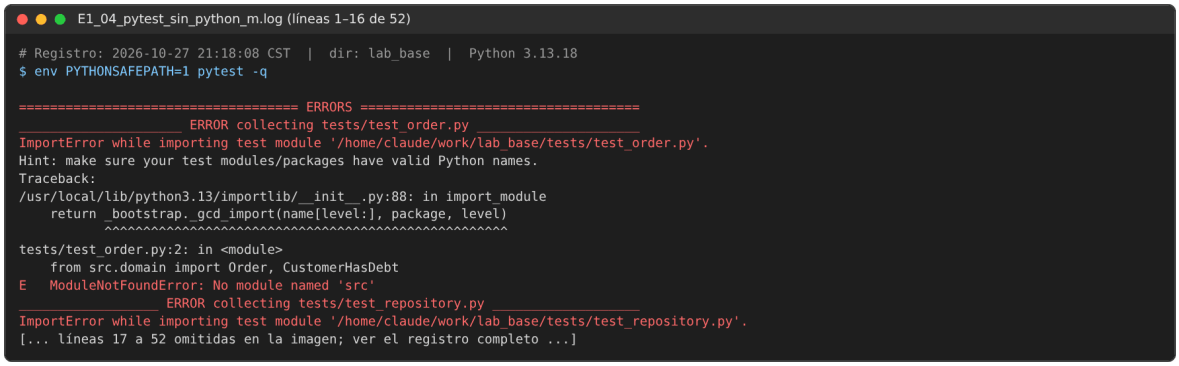


Figura 2. El comando `pytest` sin `python -m` no encuentra el paquete `src` en el proyecto original. Es la forma en que lo invocaba el workflow del profesor.

1.2 Estrategia y matriz de pruebas

Clasificamos cada caso por el comportamiento que comprueba: **éxito** cuando la operación se completa, **error** cuando la regla debe rechazarla y **límite** cuando el dato está justo en la frontera de la regla. Seguimos la pirámide de pruebas como referencia: la mayoría son unitarias de dominio y autorización, un grupo menor son de integración con SQLAlchemy y las de servicio sustituyen la pasarela con un mock. No hay pruebas extremo a extremo porque el proyecto base no tiene API ni interfaz.

La matriz del laboratorio (`matriz_pruebas_base.csv`) tiene 27 casos: 8 de éxito, 11 de error, 7 de límite y 1 riesgo documentado. Reutilizamos los casos CP01–CP15 del profesor: se leen con `csv.DictReader` en `tests/test_casos_csv.py` y también aparecen en la matriz. Las columnas «Resultado real» y «Estado» no se escribieron a mano: las genera `herramientas/generar_matrices.py` a partir del archivo JUnit XML de la ejecución final.

1.3 Pruebas unitarias y reglas de dominio

Las pruebas siguen el patrón Arrange–Act–Assert y comprueban a la vez el valor calculado y el estado final. Además de los casos pedidos (VIP de 200 con descuento de 30 y cobro de 170; regular de 200; total cero y negativo; cliente con deuda), agregamos límites que salen del propio código: total de 0.01 VIP (el 15 % redondeado es 0.00), deuda exactamente 0 frente a deuda de 0.01, el redondeo de 99.99 VIP (descuento 15.00, cobro 84.99) y la prioridad de la validación del total sobre la de deuda.

```
def test_pedido_vip_200_descuento_30_cobro_170():
    # Arrange
    order = Order(200, vip=True)
    # Act
    importe = order.confirm()
    # Assert
    assert order.discount == 30
    assert importe == 170
    assert order.status == "CONFIRMED"
```

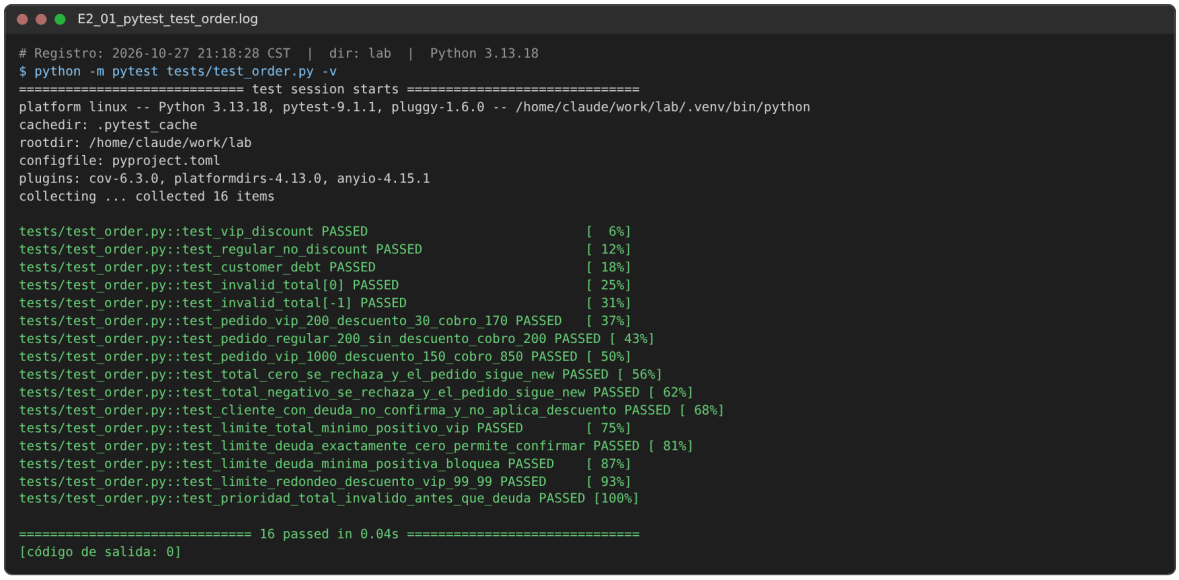


Figura 3. Pruebas unitarias del dominio (E2): 16 aprobadas, incluidas VIP 200 → 170 y regular 200 → 200.

1.4 TDD: Red, Green y Refactor

Red. Agregamos `test_cannot_confirm_twice` (el texto de la guía) y lo ejecutamos antes de cambiar `src/domain.py`. Falló con `DID NOT RAISE ValueError`, la causa esperada: la versión original volvía a confirmar el pedido y recalculaba el descuento sin protesta. Ese commit quedó en el historial como «TDD RED».

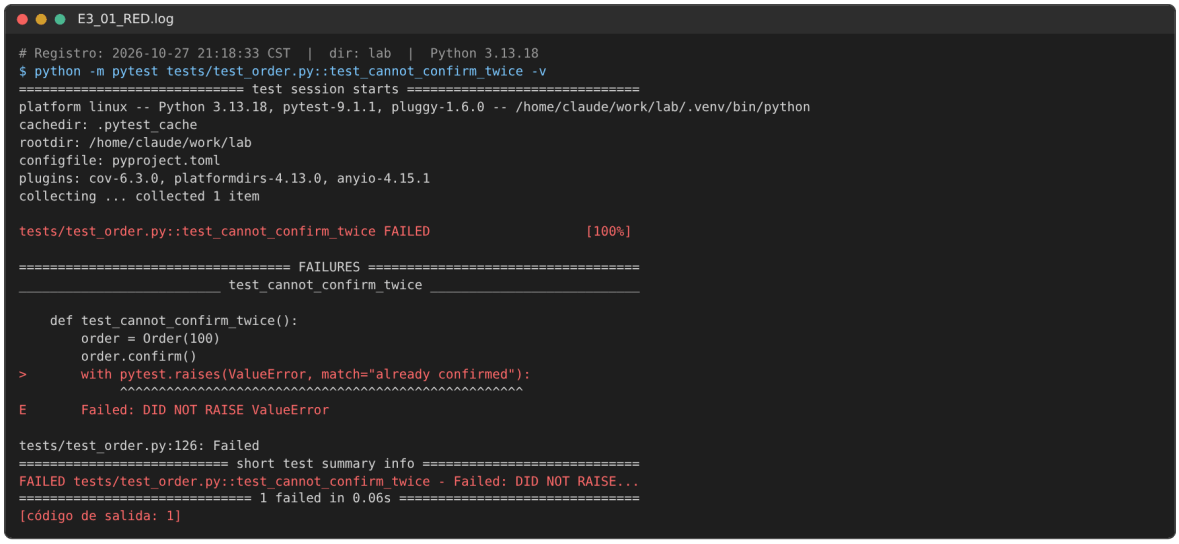
Green. El cambio mínimo fue una guarda al inicio de `confirm()`. Con ella la prueba pasó y la suite completa (25 pruebas en ese momento) siguió aprobada.

```
    def confirm(self, customer_debt=0):
+       if self.status == "CONFIRMED":
+           raise ValueError("already confirmed")
        if self.total <= 0:
```

Refactor. Extraímos las tres validaciones a `_ensure_can_confirm()` y las cadenas de estado a constantes, con el mismo orden y los mismos mensajes. Añadimos dos pruebas que protegen ese comportamiento: un

segundo intento no altera descuento ni estado, y un pedido que ya llega como `CONFIRMED` (caso CP08) se rechaza. La suite volvió a pasar completa.

La diferencia con «ejecutar pruebas después de programar» es el orden: la prueba define la regla antes de que exista el código, y el fallo inicial demuestra que la prueba sí detecta la ausencia de la regla. Una prueba escrita al final podría pasar sin haber fallado nunca y no sabríamos si verifica algo.


E3_01_RED.log
Registro: 2026-10-27 21:18:33 CST | dir: lab | Python 3.13.18
$ python -m pytest tests/test_order.py::test_cannot_confirm_twice -v
============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0 -- /home/claude/work/lab/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/claude/work/lab
configfile: pyproject.toml
plugins: cov-6.3.0, platformdirs-4.13.0, anyio-4.15.1
collecting ... collected 1 item
tests/test_order.py::test_cannot_confirm_twice FAILED [100%]
=================================== FAILURES ===================================
_____________________________ test_cannot_confirm_twice _____________________________
def test_cannot_confirm_twice():
order = Order(100)
order.confirm()
> with pytest.raises(ValueError, match="already confirmed"):
^^^

E3_07_historial_git.txt

```
1fd311f  2026-10-27 21:14  Proyecto base U5 tal como lo entrega el profesor
e77bcc2  2026-10-27 21:17  Configuración: .gitignore, pythonpath para pytest y orden de imports (ruff --fix)
dda0a74  2026-10-27 21:18  E2: pruebas unitarias del dominio (VIP 200, regular 200, errores y límites)
d9d58dd  2026-10-27 21:18  TDD RED: test_cannot_confirm_twice (falla: el código permite confirmar dos veces)
2f69919  2026-10-27 21:18  TDD GREEN: impedir confirmar un pedido ya confirmado
2750bbd  2026-10-27 21:18  TDD REFACTOR: extraer validaciones de confirm() y constantes de estado
629158e  2026-10-27 21:19  E4: pruebas de persistencia, commit, rollback y restricciones con limpieza de recursos
471a9cd  2026-10-27 21:19  E5: pago aprobado, rechazado, timeout y verificación del mock; riesgo de confirmación previa documentado (xfail)
a33ed67  2026-10-27 21:19  E6: matriz de autorización con fechas fijas (admin, propietario a 24 h, tercero, SHIPPED)
eeb547d  2026-10-27 21:20  Pruebas parametrizadas con los CSV del profesor (CP01–CP15) mediante csv.DictReader
9581091  2026-10-27 21:31  CI: workflow con Ruff, pytest (cobertura ≥ 80 %, reporte HTML), compileall y matriz 3.11/3.13; script de CI local
c93412e  2026-10-27 21:41  Documentación, matrices, evidencias y adaptación al PIA Eventos Nova
```

Figura 7. Historial de Git del laboratorio con los commits RED, GREEN y REFACTOR.

1.5 Persistencia y transacciones

Las pruebas de `tests/test_repository.py` usan SQLite en memoria; ninguna toca la base didáctica `pedidos_prueba.sqlite3` ni datos reales. Cada prueba recibe un motor nuevo por fixture, la sesión se cierra con `with` y el motor se libera con `engine.dispose()`. Verificamos: el pedido con id 2 y total 250 leído desde una **sesión nueva** después del commit (así no se lee del mapa de identidad de la sesión que escribió); un rollback que descarta solo lo no confirmado; que cerrar la sesión sin commit no persiste nada; y dos restricciones de integridad: id duplicado y total nulo.

```
def test_rollback_deshace_solo_lo_no_confirmado(engine):
    with Session(engine) as s:
        repo = SqlOrderRepository(s)
        repo.save(1, Order(120)); s.commit()        # confirmado
        repo.save(2, Order(250)); s.flush()         # enviado, transacción abierta
        assert s.scalar(select(func.count()).select_from(OrderRow)) == 2
        s.rollback()                                 # se descarta el pedido 2
    with Session(engine) as s:
        assert s.scalars(select(OrderRow.id).order_by(OrderRow.id)).all() == [1]
```

Commit y rollback. `commit()` cierra la transacción y hace permanentes sus cambios: otra sesión puede leerlos. `rollback()` descarta todo lo hecho desde el inicio de la transacción, aunque ya se haya enviado con `flush()`. El rollback no deshace lo que ya se confirmó antes, como muestra la prueba anterior.

E4_01_pytest_repository.log

```
# Registro: 2026-10-27 21:19:10 CST  |  dir: lab  |  Python 3.13.18
$ python -m pytest tests/test_repository.py -v
============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0 -- /home/claude/work/lab/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/claude/work/lab
configfile: pyproject.toml
plugins: cov-6.3.0, platformdirs-4.13.0, anyio-4.15.1
collecting ... collected 8 items

tests/test_repository.py::test_repository_commit PASSED                  [ 12%]
tests/test_repository.py::test_repository_rollback PASSED                [ 25%]
tests/test_repository.py::test_guardar_pedido_id_2_total_250_commit_y_leer_en_sesion_nueva PASSED [ 37%]
tests/test_repository.py::test_persiste_el_estado_confirmado_y_el_importe_original PASSED [ 50%]
tests/test_repository.py::test_rollback_deshace_solo_lo_no_confirmado PASSED [ 62%]
tests/test_repository.py::test_sin_commit_el_registro_no_persiste_al_cerrar_la_sesion PASSED [ 75%]
tests/test_repository.py::test_restriccion_unicidad_id_duplicado PASSED  [ 87%]
tests/test_repository.py::test_restriccion_total_no_nulo PASSED          [100%]

============================== 8 passed in 0.23s ===============================
[código de salida: 0]
```

Figura 8. Pruebas de persistencia (E4): guardado, commit, rollback y restricciones.

1.6 Servicios externos y mocks

La pasarela se sustituye por `Mock(spec_set=["charge"])`, de modo que un atributo mal escrito hace fallar la prueba y ningún cobro sale del proceso. Probamos el pago aprobado (VIP 200 devuelve 170), el rechazado (se intentó cobrar 170 y se lanzó `RuntimeError("payment rejected")`), el timeout y que la pasarela no se

llama cuando el dominio rechaza el pedido. El servicio no implementa reintentos ni circuit breaker; por eso la prueba de timeout verifica que la excepción se propaga y que hubo **un solo** intento, sin suponer políticas que no existen.

```
def test_pago_rechazado_vip_200_intenta_cobrar_170_y_lanza_runtimeerror():
    gateway = pasarela(False)
    order = Order(200, vip=True)
    with pytest.raises(RuntimeError, match="payment rejected"):
        OrderService(gateway).checkout(order)
    gateway.charge.assert_called_once_with(170)
```

Riesgo identificado. `checkout()` llama a `confirm()` antes de cobrar; si el pago se rechaza, el pedido queda `CONFIRMED` sin pago. Lo dejamos visible con una prueba `xfail(strict=True)` que expresa el comportamiento deseable; si alguien corrige el código, la prueba pasará y `strict` obligará a quitar la marca. **Propuesta (no implementada):** calcular el importe sin cambiar el estado, cobrar y confirmar solo si el cobro se aprueba, o usar un estado intermedio «pendiente de pago». No la aplicamos porque cambiaría el contrato del servicio que el laboratorio pide estudiar.

E5_01_pytest_service.log

```
# Registro: 2026-10-27 21:19:35 CST  |  dir: lab  |  Python 3.13.18
$ python -m pytest tests/test_service.py -v -rxX
============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0 -- /home/claude/work/lab/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/claude/work/lab
configfile: pyproject.toml
plugins: cov-6.3.0, platformdirs-4.13.0, anyio-4.15.1
collecting ... collected 8 items

tests/test_service.py::test_gateway_approved PASSED                      [ 12%]
tests/test_service.py::test_gateway_rejected PASSED                      [ 25%]
tests/test_service.py::test_pago_aprobado_vip_200_cobra_170 PASSED       [ 37%]
tests/test_service.py::test_pago_rechazado_vip_200_intenta_cobrar_170_y_lanza_runtimeerror PASSED [ 50%]
tests/test_service.py::test_timeout_de_la_pasarela_se_propaga_sin_reintentos PASSED [ 62%]
tests/test_service.py::test_no_se_llama_a_la_pasarela_si_el_dominio_rechaza_el_total PASSED [ 75%]
tests/test_service.py::test_no_se_cobra_dos_veces_un_pedido_ya_confirmado PASSED [ 87%]
tests/test_service.py::test_riesgo_pedido_no_deberia_quedar_confirmado_si_el_pago_se_rechaza XFAIL [100%]

=========================== short test summary info ============================
XFAIL tests/test_service.py::test_riesgo_pedido_no_deberia_quedar_confirmado_si_el_pago_se_rechaza - Riesgo documentado: checkout() confirma el ped
ido ANTES de conocer el resultado del pago; con el pago rechazado el pedido queda CONFIRMED.
======================== 7 passed, 1 xfailed in 0.04s ========================
[código de salida: 0]
```

Figura 9. Pruebas con mock de la pasarela (E5): 7 aprobadas y el riesgo documentado como XFAIL.

1.7 Seguridad y autorización

Revisamos `can_cancel()` y probamos lo que implementa: `SHIPPED` nunca se cancela, ni por el administrador; en otro estado el administrador sí puede; el propietario no administrador solo antes de cumplir 24 horas (la comparación es `<`, así que a las 24 h exactas ya no). Todas las fechas son fijas y `now` se pasa explícitamente; para el valor por defecto de `now` se sustituye el reloj del módulo con `monkeypatch`, así que ninguna prueba depende de la hora real. También cambiamos las cuatro pruebas originales para que usen fechas fijas.

Tabla 2. Matriz de autorización (E6). Pedido creado el 01/10/2026 10:00 UTC por el usuario 1

Caso	Rol	Propiedad	Estado	Antigüedad	Esperado	Real
SEG-01	Administrador	Ajeno	NEW	30 días	Permitido	Permitido
SEG-02	Administrador	Ajeno	CONFIRMED	1 h	Permitido	Permitido
SEG-03	Administrador	Ajeno	SHIPPED	1 h	Denegado	Denegado
SEG-04	Propietario	Propio	NEW	1 h	Permitido	Permitido
SEG-05	Propietario	Propio	CONFIRMED	23:59:59	Permitido	Permitido
SEG-06	Propietario	Propio	NEW	24 h exactas	Denegado	Denegado
SEG-07	Propietario	Propio	NEW	25 h	Denegado	Denegado
SEG-08	Propietario	Propio	SHIPPED	5 min	Denegado	Denegado
SEG-09	Tercero	Ajeno	NEW	5 min	Denegado	Denegado
SEG-10	Tercero	Ajeno	CONFIRMED	30 h	Denegado	Denegado

La columna «Real» corresponde a la ejecución de `test_matriz_autorizacion`, donde las diez combinaciones pasaron (Figura 10).

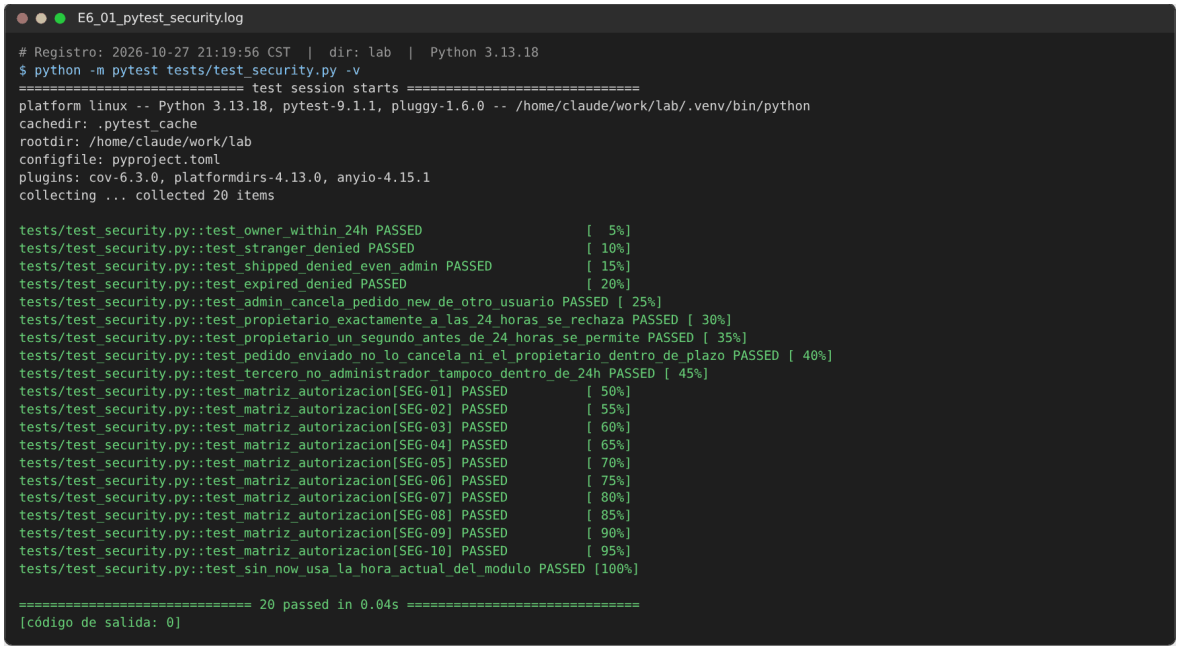


Figura 10. Pruebas de autorización (E6): 20 aprobadas.

1.8 Cobertura y Ruff

Ruff no reporta errores en `src` ni en `tests`. La ejecución final con `--cov-fail-under=80` terminó con 69 pruebas aprobadas, 1 xfail y **100 % de cobertura de líneas** en `src/`; con `--cov-branch` las 12 ramas también quedan cubiertas. No se excluyó ningún archivo. Los módulos de dominio y aplicación se ven por separado en la Tabla 3.

Tabla 3. Cobertura de la ejecución final del laboratorio (E7)

Módulo	Sentencias	Sin ejecutar	Ramas	Cobertura
src/domain.py	24	0	6	100 %
src/application.py	8	0	2	100 %
src/repository.py	18	0	0	100 %
src/security.py	8	0	4	100 %
Total	**58**	**0**	**12**	**100 %**

La cobertura de líneas solo dice qué sentencias se ejecutaron al menos una vez; no dice si las aserciones eran correctas ni si faltan casos. Nuestro propio laboratorio lo muestra: con 100 % de líneas y de ramas, el pedido sigue quedando confirmado cuando el pago se rechaza. Una prueba sin aserciones útiles podría subir la cobertura sin detectar nada.

E7_02_pytest_cobertura_html.log (líneas 82–99 de 99)

```
# Registro: 2026-10-27 21:32:32 CST  |  dir: lab  |  Python 3.13.18
$ python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html --cov-fail-under=80 --junitxml=/home/claude/work/evid/E7/junit_lab.xml
[... líneas 3 a 81 omitidas en la imagen; ver el registro completo ...]

--------- generated xml file: /home/claude/work/evid/E7/junit_lab.xml ----------
================================ tests coverage ================================
_____________ coverage: platform linux, python 3.13.18-final-0 _____________

Name                 Stmts   Miss  Cover   Missing
--------------------------------------------------
src/__init__.py          0      0   100%
src/application.py       8      0   100%
src/domain.py           24      0   100%
src/repository.py       18      0   100%
src/security.py          8      0   100%
--------------------------------------------------
TOTAL                   58      0   100%
Coverage HTML written to dir htmlcov
Required test coverage of 80% reached. Total coverage: 100.00%
======================== 69 passed, 1 xfailed in 0.58s =========================
[código de salida: 0]
```

Figura 11. Ruff y pytest con cobertura y reporte HTML (E7).

Coverage report: 100%

Files | Functions | Classes

coverage.py v7.10.7, created at 2026-10-27 21:32 -0600

File ▲	statements	missing	excluded	coverage
src/__init__.py	0	0	0	100%
src/application.py	8	0	0	100%
src/domain.py	24	0	0	100%
src/repository.py	18	0	0	100%
src/security.py	8	0	0	100%
Total	**58**	**0**	**0**	**100%**

coverage.py v7.10.7, created at 2026-10-27 21:32 -0600

Figura 12. Reporte HTML de cobertura generado en `htmlcov/index.html` (captura del navegador).

1.9 GitHub Actions y CI/CD

Conservamos las etapas del workflow del profesor y lo ajustamos: los comandos se invocan con `python -m`, se ejecuta con Python 3.11 y 3.13, el reporte HTML se publica como artefacto y el workflow también se puede lanzar a mano. Las etapas son checkout, instalación de Python, dependencias, Ruff, pytest con cobertura mínima del 80 % y `compileall`. El `.gitignore` excluye `.venv/`, `__pycache__/`, `.pytest_cache/`, `htmlcov/`, `.coverage`, `.env`, llaves y bases locales.

El YAML se validó con actionlint (sin errores). Como la sesión no tenía una cuenta de GitHub vinculada, **las tres ejecuciones reales (verde, roja y verde) siguen pendientes** y deben hacerse desde la cuenta del equipo con los comandos del `README.md`. Mientras tanto ejecutamos las mismas etapas en local, en un clon del repositorio, con `herramientas/ci_local.sh`: verde, rojo tras cambiar la aserción `170` por `171` y verde de

nuevo tras `git revert`. Esto prueba que el fallo controlado detiene las etapas, pero **no sustituye** la evidencia de GitHub. Tampoco afirmamos que haya protección de rama: se configura aparte en GitHub.

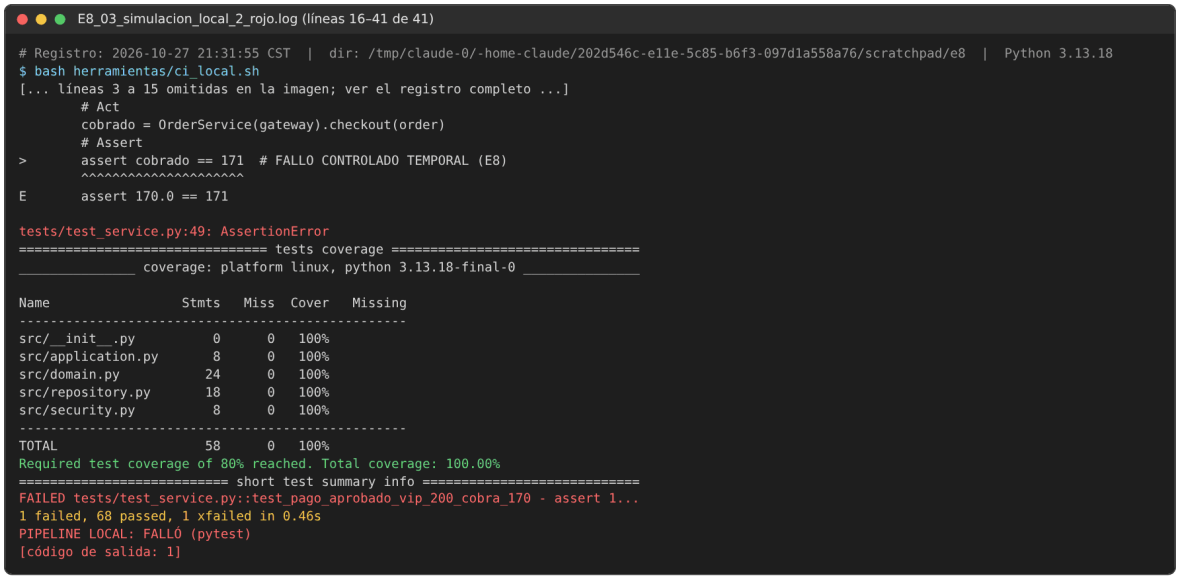


Figura 13. Simulación local del pipeline con el fallo controlado: pytest falla y el pipeline se detiene (E8, simulación local).



Figura 14. Simulación local después de revertir el fallo: todas las etapas aprobadas (E8, simulación local).

1.10 Adaptación al PIA de Eventos Nova

El ZIP de Eventos Nova trae el código real del equipo: dominio con máquinas de estado y especificaciones, servicios de aplicación, una composición en memoria y otra persistente con PostgreSQL, API FastAPI y una suite propia de 1 306 pruebas con `unittest`. Por eso las pruebas de esta unidad se escribieron **sobre ese código**, sin crear un módulo de referencia ni cambiar `src/`. Viven en `tests_u5/` y usan el mismo ayudante de flujo que la suite del equipo (`tests/apoyo_flujo.py`). Probamos la copia tal como se entregó: commit `6b501af` más los cambios locales sin confirmar que contenía.

Las reglas se tomaron del BRD corregido, las historias de usuario con sus criterios de aceptación, el glosario y el cierre de alcance aprobado del 23/10/2026. Elegimos cuatro reglas con plazos, permisos, aprobaciones y costos (Tabla 4).

Tabla 4. Reglas de Eventos Nova seleccionadas

Regla	Fuentes aprobadas	Casos
R1. Servicio adicional: al menos cinco días naturales; se verifica en orden evento → plazo → saldo y se registra solo el primer motivo	BN-004, SE-004, GL-015; US-025, CA-073, CA-075, CA-083	PIA-01 a PIA-04, PIA-03b
R2. Anticipo: al menos 30 % del costo, en tres días hábiles desde la firma; luego contacto y 24 horas naturales	BN-009, SE-010, GL-060; CA-019 a CA-021; decisión B	PIA-05 a PIA-09
R3. Cambio con menos de siete días naturales: cargo por cambio tardío según tarifa vigente, separado del costo del proveedor; sin cambios tras el inicio	BN-003, SE-003, SE-005, GL-020, GL-025; CA-039, CA-061 a CA-063	PIA-10 a PIA-13
R4. Pago de servicio adicional: 24 horas naturales desde la solicitud de pago; cierre por falta de pago	BN-004, GL-045, GL-060; CA-080, CA-082	PIA-14 a PIA-16
Integración: persistencia en PostgreSQL y dobles de los puertos externos	ADR-004; IVerificadorPagos, IConsultaDisponibilidadProveedor	PIA-17 a PIA-21

Los límites se tomaron de GL-060: los plazos en días hábiles tienen límite **exclusivo** (vencen a las 00:00 del día siguiente) y las ventanas de horas, **inclusivo**. Por eso PIA-08 prueba el seguimiento del anticipo a las 23:59:59 del martes 06/10 (todavía en plazo) y a las 00:00 del miércoles 07/10 (ya vencido), y PIA-09 y PIA-15 prueban el instante exacto de las 24 horas. La tarifa del cargo tardío no está fijada en los documentos; las pruebas usan 500.00 como dato de ejemplo, no como monto aprobado.

```
def test_pia03_cuatro_dias_no_procede_y_no_se_buscan_proveedores():
    disponibilidad = mock_disponibilidad()                # doble del puerto de proveedores
    f = contratado(disponibilidad=disponibilidad)
    dia = datetime(2026, 11, 16, 0, 0)                    # cuatro días naturales antes del
evento
    r = f.s.adicionales.registrar_solicitud_servicio_adicional(f.evento_id, f.contrato_id,
'DJ', dia)
    s = _solicitud(f, r.datos['solicitud_id'])
    assert s.estado == ESA.NO_PROCEDENTE
    assert s.motivo_cierre.causa == CausaCierreSolicitud.PLAZO_INSUFICIENTE
    disponibilidad.consultar.assert_not_called()          # CA-083: no continúa con la
búsqueda
```

Persistencia y mocks. PIA-17 y PIA-18 ejecutan el caso con la composición persistente y lo leen desde otro contenedor, como si la aplicación se hubiera reiniciado. PIA-19 guarda una solicitud y revierte la unidad de trabajo, y guarda otra y la confirma: solo la segunda existe en PostgreSQL. PIA-20 sustituye el verificador de pagos (la verificación bancaria) con un mock que primero rechaza y luego aprueba, y PIA-21 comprueba que primero se consulta a los proveedores actuales y después a las alternativas.

Resultado. 21 pruebas aprobadas y 1 `xfail`, Ruff sin errores. Las pruebas de PostgreSQL corrieron con `EVENTOSNOVA_REQUIERE_BD=1`, que convierte la falta de base en fallo, así que no se omitieron. La cobertura de `src/` con solo `tests_u5` es 50.4 %, lo esperable para 22 pruebas enfocadas en cuatro reglas. Con la suite del equipo más `tests_u5` la cobertura de sentencias es 98.82 % y la de ramas 95.85 %.

Tabla 5. Diferencias entre la guía de la actividad y la documentación aprobada de Eventos Nova

Punto de la guía	Documentación aprobada	Decisión
Anticipo del 30 %	Como mínimo el 30 % (BN-009, CA-019, decisión B)	Se prueba como mínimo (PIA-06, PIA-07)
Pago final siete días antes del evento	Saldo tres días hábiles antes (BN-005)	No se prueba la regla de la guía
Cambio con < 7 días: «cargo fijo»	Cargo según la tarifa vigente al registrar, solo si el cambio es viable, aceptado y realizado (SE-003, SE-005, GL-025)	La tarifa es un dato; 500.00 es ejemplo
Administración autoriza «cuando corresponda»	Autorización obligatoria del servicio adicional tras la aprobación del Cliente (BN-004)	Se sigue el BRD
Saldo vencido «antes de procesar» un servicio adicional	Se verifica al final: evento → plazo → saldo (SE-004, GL-015)	Se prueba el orden aprobado (PIA-04)
«No reembolso»	Política incluida y comunicada (BN-009); existen dos devoluciones aprobadas (SE-012 y decisión A)	Las devoluciones no se prueban en U5

E9_03_pytest_tests_u5.log

```
# Registro: 2026-10-27 21:28:28 CST  |  dir: pia/Eventosnova  |  Python 3.13.18
$ python -m pytest tests_u5 -v -rxX -p no:cacheprovider
============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0 -- /home/claude/work/pia/.venv/bin/python
rootdir: /home/claude/work/pia/Eventosnova
plugins: cov-6.3.0, platformdirs-4.13.0, anyio-4.15.1
collecting ... collected 22 items

tests_u5/test_u5_integracion_persistencia_y_mocks.py::test_pia17_anticipo_del_40_por_ciento_persiste_y_se_lee_tras_reiniciar PASSED [  4%]
tests_u5/test_u5_integracion_persistencia_y_mocks.py::test_pia18_solicitud_no_procedente_por_plazo_persiste_con_su_motivo PASSED [  9%]
tests_u5/test_u5_integracion_persistencia_y_mocks.py::test_pia19_rollback_de_la_unidad_de_trabajo_no_deja_rastro_y_commit_si PASSED [ 13%]
tests_u5/test_u5_integracion_persistencia_y_mocks.py::test_pia20_verificador_de_pagos_simulado_rechaza_y_luego_aprueba PASSED [ 18%]
tests_u5/test_u5_integracion_persistencia_y_mocks.py::test_pia21_consulta_primero_a_proveedores_actuales_y_despues_alternativas PASSED [ 22%]
tests_u5/test_u5_r1_servicio_adicional_plazo.py::test_pia01_solicitud_con_diez_dias_procede_y_consulta_proveedores PASSED [ 27%]
tests_u5/test_u5_r1_servicio_adicional_plazo.py::test_pia02_limite_exactamente_cinco_dias_naturales_procede PASSED [ 31%]
tests_u5/test_u5_r1_servicio_adicional_plazo.py::test_pia03_cuatro_dias_no_procede_y_no_se_buscan_proveedores PASSED [ 36%]
tests_u5/test_u5_r1_servicio_adicional_plazo.py::test_pia03b_solicitud_no_procedente_no_debe_consultar_proveedores_al_reintentar XFAIL [ 40%]
tests_u5/test_u5_r1_servicio_adicional_plazo.py::test_pia04_evento_cancelado_se_informa_primero_aunque_falten_menos_de_cinco_dias PASSED [ 45%]
tests_u5/test_u5_r2_anticipo.py::test_pia05_anticipo_exacto_del_30_por_ciento_confirma_la_contratacion PASSED [ 50%]
tests_u5/test_u5_r2_anticipo.py::test_pia06_un_centavo_menos_del_30_por_ciento_no_confirma_hasta_completar PASSED [ 54%]
tests_u5/test_u5_r2_anticipo.py::test_pia07_anticipo_del_40_por_ciento_cubre_el_requisito_sin_devolucion_ni_abono_al_saldo PASSED [ 59%]
tests_u5/test_u5_r2_anticipo.py::test_pia08_seguimiento_solo_despues_del_limite_exclusivo_de_tres_dias_habiles PASSED [ 63%]
tests_u5/test_u5_r2_anticipo.py::test_pia09_sin_anticipo_24_horas_despues_del_contacto_se_cierra_sin_contratacion PASSED [ 68%]
tests_u5/test_u5_r3_cambio_tardio.py::test_pia10_limite_siete_dias_exactos_sin_cargo_tardio PASSED [ 72%]
tests_u5/test_u5_r3_cambio_tardio.py::test_pia11_seis_dias_genera_cargo_tardio_y_costo_del_proveedor_por_separado PASSED [ 77%]
tests_u5/test_u5_r3_cambio_tardio.py::test_pia12_cambio_tardio_no_viable_no_procede_y_no_cobra_el_cargo PASSED [ 81%]
tests_u5/test_u5_r3_cambio_tardio.py::test_pia13_no_se_admiten_cambios_una_vez_iniciado_el_evento PASSED [ 86%]
tests_u5/test_u5_r4_pago_servicio_adicional_24h.py::test_pia14_pago_dentro_de_las_24_horas_permite_confirmar_el_servicio PASSED [ 90%]
tests_u5/test_u5_r4_pago_servicio_adicional_24h.py::test_pia15_limite_pago_en_la_hora_24_exacta_verificado_despues_no_cierra PASSED [ 95%]
tests_u5/test_u5_r4_pago_servicio_adicional_24h.py::test_pia16_sin_pago_un_segundo_despues_del_limite_se_cierra_por_falta_de_pago PASSED [100%]

========================= short test summary info ==========================
XFAIL tests_u5/test_u5_r1_servicio_adicional_plazo.py::test_pia03b_solicitud_no_procedente_no_debe_consultar_proveedores_al_reintentar - Hallazgo U
5-H1: evaluar_disponibilidad_servicio_adicional() consulta al puerto de proveedores ANTES de que el dominio rechace la transición de una solicitud
que ya es No procedente (CA-083 indica no continuar con la búsqueda).
======================== 21 passed, 1 xfailed in 1.45s =========================
[código de salida: 0]
```

Figura 15. Pruebas U5 sobre el código real de Eventos Nova (E9): 21 aprobadas y 1 XFAIL.

1.11 Uso responsable de IA

Usamos Claude (Anthropic) en una sesión de trabajo con acceso a archivos y terminal. El equipo le dio un solo prompt: las instrucciones de esta actividad, con el material de la unidad y el ZIP de Eventos Nova. La bitácora (`bitacora_uso_IA.md`) registra 11 propuestas de la IA: qué se propuso, cómo se revisó contra los documentos y la ejecución, y qué se decidió. No usamos el prompt sugerido de la guía y no hubo otras consultas.

Dos ejemplos de sugerencias corregidas. La IA escribió una prueba, `test_el_servicio_no_recibe_la_deuda_del_cliente`, que solo confirmaba un pedido con la pasarela simulada: no verificaba ninguna regla y la descartamos. En PIA-13 propuso la aserción `assert … == [] if`

`hasattr(…) else True`, que podía pasar sin comprobar nada; la reemplazamos por `assert … == []`. También rechazamos la lista inicial de reglas de la guía cuando no coincidía con el BRD (Tabla 5). La revisión del equipo sobre cada entrada de la bitácora debe registrarse antes de entregar.

2. Ejemplos de funcionamiento

Pedido VIP con pago rechazado (laboratorio). Un pedido VIP de 200 se envía a `checkout()` con la pasarela simulada respondiendo `False`. El mock registra una llamada con 170 (200 menos el 15 %) y el servicio lanza `RuntimeError("payment rejected")`. El pedido queda `CONFIRMED`, que es el riesgo de la sección 1.6.

Propietario a las 24 horas (laboratorio). Un pedido creado el 01/10/2026 a las 10:00 se intenta cancelar el 02/10 a las 10:00. `can_cancel()` devuelve `False`; un segundo antes devuelve `True`.

Anticipo de 2 999.99 (Eventos Nova). Con un costo de 10 000.00, el pago de 2 999.99 queda verificado como pago parcial, el anticipo sigue Pendiente y el contrato sigue Firmado. Al registrar 0.01 más el anticipo se liquida y la contratación se confirma.

Cambio tardío (Eventos Nova). Un cambio pedido el 14/11/2026 a las 18:00 para un evento del 20/11 a las 18:00 tiene seis días naturales: al comunicar la solicitud de pago se crean dos obligaciones independientes, el cargo por cambio tardío (500.00) y el costo del proveedor (800.00). El mismo cambio pedido el 13/11 solo genera el costo del proveedor.

3. Pruebas y validaciones

Tabla 6. Comandos ejecutados y resultados verificables

Alcance	Comando	Resultado
Laboratorio	`python -m ruff check src tests`	All checks passed
Laboratorio	`python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html --cov-fail-under=80`	69 aprobadas, 1 xfail; cobertura 100 %; `htmlcov/index.html` generado
Laboratorio	`python -m pytest -q --cov=src --cov-branch`	Líneas y ramas: 100 %
Laboratorio	`python -m compileall -q src`	Código de salida 0
Laboratorio	`actionlint .github/workflows/ci.yml`	0 errores
Eventos Nova	`python -m pytest tests_u5 -v -rxX`	21 aprobadas, 1 xfail (PostgreSQL 16 local)
Eventos Nova	`python -m ruff check tests_u5`	All checks passed
Eventos Nova	`coverage run -m unittest discover -s tests` + `coverage run -a -m pytest tests_u5`	Sentencias 98.82 %, ramas 95.85 % (mínimo 80 %: cumple)
Eventos Nova	`coverage run -m unittest discover -s tests` (suite del equipo)	1 306 pruebas, 1 fallo de entorno (sección 6)

E9_06_cobertura_suite_equipo_mas_u5.log

```
# Registro: 2026-10-27 21:30:35 CST  |  dir: pia/Eventosnova  |  Python 3.13.18
$ sh -c python -m coverage report --skip-covered | tail -25; python -m coverage html -q -d htmlcov_pia_completo; python -m coverage json -q -o html
cov_pia_completo/coverage.json; python scripts/verificar_cobertura.py htmlcov_pia_completo/coverage.json --minimo 80
src/domain/contrato.py                                    85      1     22      1  98.1%   71
src/domain/estado_cuenta.py                              318      2     86      3  98.8%   266, 285, 292->289
src/domain/evento.py                                     179      4     58      4  96.6%   163, 171, 260, 262
src/domain/servicios_dominio.py                           50      0      8      1  98.3%   89->91
src/ejecucion_paquete/__main__.py                         19      1      8      1  92.6%   26
src/ejecucion_paquete/casos.py                            81      1     14      1  97.9%   61
src/ejecucion_paquete/evidencias.py                      108      2     28      2  97.1%   41, 45
src/ejecucion_paquete/oraculo.py                          30      1      6      1  94.4%   50
src/infrastructure/persistencia/codec_eventos.py          81      0     42      1  99.2%   82->85
src/infrastructure/persistencia/composicion_persistente.py      51      3     12      3  90.5%   41->43, 44->51, 49, 75-76
src/infrastructure/persistencia/conexion.py               92      2     16      0  98.1%   73-74
src/infrastructure/persistencia/outbox_sql.py            127      3     24      1  97.4%   57, 146, 148
src/infrastructure/persistencia/preparacion.py            63      3     10      3  91.8%   42, 67, 75
src/infrastructure/persistencia/trabajador_outbox.py      45      1     12      1  96.5%   63
src/infrastructure/persistencia/unidad_trabajo_sql.py     48      1     12      0  98.3%   40
src/infrastructure/unidad_trabajo.py                      71      2     20      0  95.6%   65-66
src/paquete_pia/__main__.py                               28      1      6      1  94.1%   42
src/paquete_pia/errores.py                                25      1      6      1  93.5%   34
src/paquete_pia/formatos.py                               56      2     18      0  97.3%   51-52
src/paquete_pia/lector.py                                 99      5     50      2  95.3%   74-76, 79-80, 143->exit
src/paquete_pia/manifiesto.py                             58      3     24      2  93.9%   40, 73-74
-----------------------------------------------------------------------------------------------------
TOTAL                                                   5443     64   1060     40  98.3%

44 files skipped due to complete coverage.
Cobertura de sentencias: 98.82 % (5379 de 5443; sin cubrir 64)
Cobertura de ramas:      95.85 % (1016 de 1060)
OK: cumple el mínimo de 80 %.
[código de salida: 0]
```

Figura 16. Cobertura de Eventos Nova con la suite del equipo y las pruebas U5 (E9).

4. Matrices y resultados de prueba

Las dos matrices completas están en el Anexo A y en los CSV del entregable. En ambas, el resultado real coincide con el esperado en todos los casos menos los dos `xfail`, que son diferencias documentadas a propósito.

Tabla 7. Resumen de las matrices

Matriz	Casos	Éxito	Error	Límite	Aprobados	XFAIL
Laboratorio base (`matriz_pruebas_base.csv`)	27	8	12	7	26	1 (LAB-27, riesgo)
Eventos Nova (`matriz_pruebas_PIA.csv`)	22	7	9	6	21	1 (PIA-03b, hallazgo)

En la matriz de Eventos Nova, PIA-20 cuenta como error porque empieza con un rechazo del verificador y termina en éxito; las demás categorías siguen la columna «Categoría» del CSV.

5. Evidencias E1–E9

Cada evidencia tiene en `evidencias/` el registro de texto con fecha, comando y código de salida, y su imagen. Las imágenes de terminal se generaron a partir de esos registros sin cambiar el texto; las de cobertura HTML son capturas del navegador.

Tabla 8. Estado de las evidencias

ID	Objetivo	Comando o acción	Resultado observado	Requisito	Estado
E1	Instalación y prueba inicial	`python -m pytest -v`; cobertura; Ruff	13/13; 98 %; 7 avisos de Ruff; fallo de `pytest` directo	5.1	Completa
E2	VIP y regular; límites; CSV	`pytest tests/test_order.py`, `test_casos_csv.py`	16/16 y 15/15	5.2	Completa
E3	TDD	RED, GREEN, suite, refactor	RED por `DID NOT RAISE`; GREEN; suite aprobada	5.3	Completa
E4	Persistencia	`pytest tests/test_repository.py`	8/8	5.4	Completa
E5	Mocks	`pytest tests/test_service.py -rxX`	7 aprobadas, 1 xfail	5.5	Completa
E6	Autorización	`pytest tests/test_security.py`	20/20	5.6	Completa
E7	Ruff, cobertura, HTML	Ruff, pytest-cov, compileall	Sin errores; 100 %; HTML generado	5.7	Completa
E8	GitHub Actions	actionlint; simulación local	YAML válido; local verde → rojo → verde	5.8	**Pendiente en GitHub**
E9	PIA Eventos Nova	`pytest tests_u5`; cobertura combinada	21 + 1 xfail; 98.82 %	6	Completa (local)



Figura 17. E1: Ruff sobre el código original reporta 7 errores corregibles.

E2_02_pytest_casos_csv.log

```
# Registro: 2026-10-27 21:20:16 CST  |  dir: lab  |  Python 3.13.18
$ python -m pytest tests/test_casos_csv.py -v
============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0 -- /home/claude/work/lab/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/claude/work/lab
configfile: pyproject.toml
plugins: cov-6.3.0, platformdirs-4.13.0, anyio-4.15.1
collecting ... collected 15 items

tests/test_casos_csv.py::test_confirmar_segun_csv[CP01] PASSED           [  6%]
tests/test_casos_csv.py::test_confirmar_segun_csv[CP02] PASSED           [ 13%]
tests/test_casos_csv.py::test_confirmar_segun_csv[CP03] PASSED           [ 20%]
tests/test_casos_csv.py::test_confirmar_segun_csv[CP04] PASSED           [ 26%]
tests/test_casos_csv.py::test_confirmar_segun_csv[CP05] PASSED           [ 33%]
tests/test_casos_csv.py::test_confirmar_segun_csv[CP06] PASSED           [ 40%]
tests/test_casos_csv.py::test_confirmar_segun_csv[CP07] PASSED           [ 46%]
tests/test_casos_csv.py::test_confirmar_segun_csv[CP08] PASSED           [ 53%]
tests/test_casos_csv.py::test_confirmar_segun_csv[CP09] PASSED           [ 60%]
tests/test_casos_csv.py::test_confirmar_segun_csv[CP10] PASSED           [ 66%]
tests/test_casos_csv.py::test_confirmar_segun_csv[CP11] PASSED           [ 73%]
tests/test_casos_csv.py::test_confirmar_segun_csv[CP12] PASSED           [ 80%]
tests/test_casos_csv.py::test_cp13_pago_rechazado_segun_csv PASSED       [ 86%]
tests/test_casos_csv.py::test_cp14_cp15_commit_y_rollback_segun_csv[CP14-True-True] PASSED [ 93%]
tests/test_casos_csv.py::test_cp14_cp15_commit_y_rollback_segun_csv[CP15-False-False] PASSED [100%]

============================== 15 passed in 0.27s ==============================
[código de salida: 0]
```

Figura 18. E2: casos CP01–CP15 del profesor leídos desde CSV.

E3_02_GREEN_cambio.diff

```
diff --git a/src/domain.py b/src/domain.py
index f63c2de..b59e450 100644
--- a/src/domain.py
+++ b/src/domain.py
@@ -12,6 +12,8 @@ class Order:
     discount: float = 0.0

     def confirm(self, customer_debt=0):
+        if self.status == "CONFIRMED":
+            raise ValueError("already confirmed")
         if self.total <= 0:
             raise ValueError("total must be positive")
         if customer_debt > 0:
```

Figura 19. E3: cambio mínimo de la fase GREEN (diff).

Coverage for **src/domain.py**: 100%

24 statements | 24 run | 0 missing | 0 excluded

« prev ^ index » next coverage.py v7.10.7, created at 2026-10-27 21:32 -0600

```python
from dataclasses import dataclass

NEW = "NEW"
CONFIRMED = "CONFIRMED"
VIP_DISCOUNT_RATE = 0.15


class CustomerHasDebt(Exception):
    pass


@dataclass
class Order:
    total: float
    vip: bool = False
    status: str = NEW
    discount: float = 0.0

    def confirm(self, customer_debt=0):
        self._ensure_can_confirm(customer_debt)
        self.discount = round(self.total * VIP_DISCOUNT_RATE, 2) if self.vip else 0.0
        self.status = CONFIRMED
        return self.total - self.discount

    def _ensure_can_confirm(self, customer_debt):
        """Reglas previas a confirmar, en el mismo orden que la versión GREEN."""
        if self.status == CONFIRMED:
            raise ValueError("already confirmed")
        if self.total <= 0:
            raise ValueError("total must be positive")
        if customer_debt > 0:
            raise CustomerHasDebt("outstanding debt")
```

« prev ^ index » next coverage.py v7.10.7, created at 2026-10-27 21:32 -0600

Figura 20. E7: detalle de `src/domain.py` en el reporte HTML; todas las líneas ejecutadas.

E8_01_actionlint.log

```
# Registro: 2026-10-27 21:20:37 CST  |  dir: lab  |  Python 3.13.18
$ /home/claude/work/tools/actionlint -verbose .github/workflows/ci.yml
verbose: Linting .github/workflows/ci.yml
verbose: Using project at /home/claude/work/lab
verbose: Found 0 parse errors in 0 ms for .github/workflows/ci.yml
verbose: Rule "shellcheck" was disabled: exec: "shellcheck": executable file not found in $PATH
verbose: Rule "pyflakes" was disabled: exec: "pyflakes": executable file not found in $PATH
verbose: Found total 0 errors in 0 ms for .github/workflows/ci.yml
[código de salida: 0]
```

Figura 21. E8: validación del workflow con actionlint.

Eventos Nova - cobertura de pruebas: 50.4%

Files | Functions | Classes

filter... hide covered

coverage.py v7.16.2, created at 2026-10-27 21:28 -0600

	Statements				Branches			Total
File ▲	*coverage*	*statements*	*missing*	*excluded*	*coverage*	*branches*	*partial*	*coverage*
src/__init__.py	100.0%	0	0	0	100.0%	0	0	100.0%
src/api/__init__.py	100.0%	0	0	0	100.0%	0	0	100.0%
src/api/dependencias.py	0.0%	11	11	0	100.0%	0	0	0.0%
src/api/errores.py	0.0%	52	52	0	0.0%	4	0	0.0%
src/api/esquemas_operaciones.py	0.0%	99	99	0	100.0%	0	0	0.0%
src/api/esquemas.py	0.0%	42	42	0	0.0%	2	0	0.0%
src/api/main.py	0.0%	59	59	0	0.0%	10	0	0.0%
src/api/rutas_comunes.py	0.0%	24	24	0	0.0%	4	0	0.0%
src/api/rutas_consultas.py	0.0%	42	42	0	100.0%	0	0	0.0%
src/api/rutas_contratacion.py	0.0%	34	34	0	100.0%	0	0	0.0%
src/api/rutas_eventos.py	0.0%	36	36	0	100.0%	0	0	0.0%
src/api/rutas_pagos.py	0.0%	40	40	0	100.0%	0	0	0.0%
src/api/rutas_propuestas.py	0.0%	18	18	0	100.0%	0	0	0.0%
src/api/rutas_solicitudes.py	0.0%	77	77	0	100.0%	0	0	0.0%
src/application/__init__.py	100.0%	0	0	0	100.0%	0	0	100.0%
src/application/cqrs.py	71.4%	343	98	0	0.0%	48	0	62.7%
src/application/puertos.py	100.0%	29	0	95	100.0%	0	0	100.0%
src/application/reacciones.py	82.5%	114	20	0	44.7%	38	5	73.0%
src/application/resultados.py	100.0%	15	0	0	100.0%	0	0	100.0%
src/application/servicios.py	59.7%	668	269	0	41.7%	108	29	57.2%
src/domain/__init__.py	100.0%	0	0	0	100.0%	0	0	100.0%
src/domain/base.py	100.0%	13	0	0	100.0%	0	0	100.0%

Figura 22. E9: reporte HTML de cobertura de las pruebas U5 sobre Eventos Nova.

6. Incidencias y correcciones

Tabla 9. Incidencias observadas

ID	Incidencia	Causa	Solución o estado
I-01	`pytest` sin `python -m` no encuentra `src`	La raíz del proyecto no estaba en `sys.path`	`pythonpath = ["."]` en `pyproject.toml`
I-02	Ruff reporta 7 errores en el código original	`ruff>=0.6` instala la versión más reciente, que ordena imports	`ruff check --fix` (solo imports); suite completa aprobada
I-03	`ResourceWarning` por conexiones SQLite abiertas	Las pruebas no liberaban el motor	Fixtures con `dispose()` y advertencias convertidas en error
I-04	Sin acceso a PyPI en el entorno	Política de red del entorno	Dependencias instaladas desde GitHub y desde el entorno del equipo; mismas versiones mayores
I-05	Ruff señala `DTZ001` en `tests_u5`	El dominio de Eventos Nova usa fechas locales sin zona (TP-06)	`tests_u5/ruff.toml` ignora esa regla y lo justifica
I-PIA-1	`test_openapi_exportada` falla en la suite del equipo	Con Pydantic 2.13.5 el esquema agrega un `pattern` a 8 campos decimales; el archivo del equipo se exportó con Pydantic 2.14.1	No es un cambio del código; confirmar en el entorno del equipo
U5-H1	Se consulta a proveedores con una solicitud ya No procedente	El servicio llama al puerto antes de que el dominio valide el estado	Hallazgo documentado con `xfail`; corrección propuesta, no aplicada
U5-O1	Días naturales por fecha o por 24 h	Los documentos no lo precisan; el código usa fechas de calendario	Ambigüedad registrada (PIA-02)
E8	Sin ejecuciones reales de GitHub Actions	Sesión sin cuenta de GitHub vinculada	Pendiente del equipo

Durante el análisis también observamos que, con el saldo exigible tres días hábiles antes del evento y el mínimo de cinco días naturales del servicio adicional, cuando el saldo ya venció nunca quedan cinco días naturales (como máximo cuatro). En una solicitud nueva de servicio adicional se informa siempre el motivo de plazo antes que el de saldo vencido. Es consecuencia del orden aprobado, no un defecto, y lo dejamos anotado para el equipo.

7. Preguntas y respuestas

¿Por qué una prueba unitaria no necesita base de datos? Porque verifica una regla en aislamiento. `Order.confirm()` no depende de SQLAlchemy; si la prueba usara una base, un fallo podría venir de la conexión y no de la regla.

¿Qué distingue una prueba de integración? Ejercita varias piezas juntas: en el laboratorio, el repositorio con SQLAlchemy y SQLite; en Eventos Nova, los servicios con la composición persistente y PostgreSQL.

¿Qué es TDD? Escribir primero una prueba que falla (Red), escribir el mínimo código para que pase (Green) y mejorar el diseño sin cambiar el comportamiento (Refactor). El fallo inicial prueba que la prueba detecta la ausencia de la regla.

¿Para qué sirve un mock? Para sustituir un servicio externo y controlar su respuesta. Permite probar el rechazo o el timeout de un pago sin cobrar y verificar con qué datos se llamó.

¿Qué diferencia hay entre commit y rollback? `commit()` hace permanentes los cambios de la transacción; `rollback()` los descarta. Ninguno afecta a lo confirmado en transacciones anteriores.

¿Autenticación y autorización son lo mismo? No. La autenticación establece quién es el usuario; la autorización decide qué puede hacer. `can_cancel()` es una política de autorización: recibe la identidad ya establecida.

¿Qué demuestra un 80 % de cobertura? Que al menos el 80 % de las líneas medidas se ejecutaron. No demuestra que el 80 % de los defectos se haya encontrado.

¿Qué aporta CI/CD? Ejecuta las mismas verificaciones en cada push o pull request. Si Ruff, las pruebas o la cobertura fallan, el cambio queda marcado en rojo; bloquearlo exige además reglas de protección de la rama.

Reflexión: ¿qué defecto de negocio sería más costoso en producción y qué prueba lo evitaría?

En el laboratorio, el defecto más costoso sería cobrar o confirmar mal un pedido: hoy `checkout()` deja el pedido en CONFIRMED aunque la pasarela rechace el pago, de modo que se podría enviar mercancía sin haber cobrado. No es un defecto que hayamos introducido ni que una prueba haya descubierto por accidente: lo señala la guía y lo dejamos visible con una prueba xfail que exige que el pedido no quede confirmado tras el rechazo. En Eventos Nova, el riesgo equivalente es contar mal un plazo. Si el anticipo se considerara vencido un minuto antes, se cerraría una contratación que el Cliente pagó a tiempo; si la ventana de 24 horas del servicio adicional se tomara como exclusiva, se cerraría por falta de pago una solicitud pagada en el último instante. Las pruebas de límite PIA-08, PIA-09 y PIA-15, que fijan el instante exacto con fechas fijas, evitarían que una refactorización cambie un «<» por un «<=» sin que nadie lo note. Aprendimos que una cobertura del 100 % no basta: hacen falta aserciones sobre el estado y sobre los límites.

8. Conclusión general

Completamos las pruebas del laboratorio con 69 aprobadas, 1 riesgo documentado, Ruff sin errores y 100 % de cobertura de líneas y ramas; aplicamos TDD con evidencia real de RED y GREEN y dejamos un workflow validado. En Eventos Nova probamos cuatro reglas aprobadas sobre el código real, incluida persistencia en PostgreSQL y dobles de los servicios externos, y encontramos un hallazgo y una ambigüedad que no aparecían en la suite existente. Lo que falta es externo a este entorno: las ejecuciones de GitHub Actions y la integración de `tests_u5` en el repositorio del equipo.

9. Conclusiones individuales

(Texto propuesto. Cada integrante debe revisarlo y ajustarlo con sus propias palabras antes de entregar.)

Lesly Adamaris Cotlame Nieto. Entendí que la fase RED no es un paso de trámite: ver fallar la prueba por la causa correcta fue lo que me dio confianza en que la guarda de doble confirmación estaba realmente probada.

Christopher Bethancourt Contreras. Aprendí a probar persistencia sin depender de una base de producción y a distinguir con pruebas lo que hace un commit de lo que deshace un rollback, incluso después de un flush.

Francisco Mora Ruiz. Me quedo con que los límites se deben confirmar en los documentos: en Eventos Nova, los días hábiles vencen a las 00:00 y las ventanas de horas incluyen el instante final, y eso cambió el resultado esperado de varios casos.

10. Repositorio de GitHub

Pendiente. El repositorio local está listo, con historial de commits, workflow y README. La URL se anotará cuando el equipo lo publique y ejecute las tres corridas de E8. El repositorio de Eventos Nova tampoco tiene remoto configurado.

Bibliografía

Astral Software. (s.f.). *Ruff* [Documentación]. https://docs.astral.sh/ruff/

Batchelder, N. (s.f.). *Coverage.py* [Documentación]. https://coverage.readthedocs.io/

Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley.

Componentes de Lógica de Negocios. (2026). *Laboratorio guiado Unidad 5: Integración, pruebas y calidad* [Material del curso]. Facultad de Ingeniería Mecánica y Eléctrica, UANL.

Cotlame Nieto, L. A., Bethancourt Contreras, C., & Mora Ruiz, F. (2026). *Reporte BRD corregido, historias de usuario y glosario de Eventos Nova* [Documentos no publicados]. FIME, UANL.

GitHub. (s.f.). *Workflow syntax*. GitHub Docs. https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax

Meszaros, G. (2007). *xUnit test patterns: Refactoring test code*. Addison-Wesley.

pytest-dev. (s.f.). *How to use skip and xfail to deal with tests that cannot succeed*. pytest documentation. https://docs.pytest.org/en/stable/how-to/skipping.html

pytest-dev. (s.f.). *pytest-cov* [Repositorio de código]. GitHub. https://github.com/pytest-dev/pytest-cov

Python Software Foundation. (s.f.). *unittest.mock — mock object library*. Python 3 documentation. https://docs.python.org/3/library/unittest.mock.html

SQLAlchemy. (s.f.). *Transactions and connection management*. SQLAlchemy 2.0 Documentation. https://docs.sqlalchemy.org/en/20/orm/session_transaction.html

Anexos

Anexo A. Matriz completa del laboratorio base

Tabla 10. Matriz de pruebas del laboratorio base (27 casos)

ID	Regla	Entrada	Esperado	Categoría	Estado	Ev.
LAB-01	Pedido regular sin descuento	CP01: total 120, regular	CONFIRMED, cobro 120.00	Éxito	PASSED	E2
LAB-02	Descuento VIP 15 %	total 200, VIP	Descuento 30; cobro 170; estado CONFIRMED	Éxito	PASSED	E2
LAB-03	Pedido regular sin descuento	total 200, regular	Descuento 0; cobro 200; estado CONFIRMED	Éxito	PASSED	E2
LAB-04	Descuento VIP 15 %	total 1000, VIP (CP06)	Descuento 150; cobro 850	Éxito	PASSED	E2
LAB-05	Persistencia con commit	id 2, total 250	Registro id 2, total 250, estado NEW visible	Éxito	PASSED	E4
LAB-06	Cobro con pasarela (aprobado)	total 200, VIP	Devuelve 170; charge() llamado una vez con 170; CONFIRMED	Éxito	PASSED	E5
LAB-07	Cancelación por administrador	admin, estado NEW	Permitido	Éxito	PASSED	E6
LAB-08	Cancelación por propietario < 24 h	propietario no admin	Permitido	Éxito	PASSED	E6
LAB-09	Total debe ser positivo	total 0	ValueError("total must be positive"); estado sigue NEW	Error	PASSED	E2
LAB-10	Total debe ser positivo	total -10	ValueError; estado sigue NEW	Error	PASSED	E2
LAB-11	Cliente con deuda no confirma	total 500, deuda 250 (CP11)	CustomerHasDebt; estado NEW; sin descuento	Error	PASSED	E2
LAB-12	No confirmar dos veces (TDD)	total 100	ValueError("already confirmed")	Error	PASSED	E3
LAB-13	Cobro con pasarela (rechazado)	total 200, VIP	RuntimeError("payment rejected"); charge() llamado con 170	Error	PASSED	E5
LAB-14	Timeout de la pasarela	total 120	TimeoutError propagado; un solo intento (sin reintentos implementados)	Error	PASSED	E5
LAB-15	Unicidad de id	id 1 repetido	IntegrityError; el registro original no cambia	Error	PASSED	E4
LAB-16	Total no nulo	total None	IntegrityError; nada se guarda	Error	PASSED	E4
LAB-17	Rollback de cambios pendientes	id 1 y 2	Solo existe el pedido 1	Error	PASSED	E4
LAB-18	Tercero no autorizado	usuario 2, no admin	Denegado	Error	PASSED	E6
LAB-19	SHIPPED nunca se cancela	administrador	Denegado	Error	PASSED	E6
LAB-20	Total mínimo positivo	total 0.01 (CP07)	Descuento 0.00; cobro 0.01; CONFIRMED	Límite	PASSED	E2
LAB-21	Deuda igual a cero	total 120, deuda 0	Cobro 120 (permitido)	Límite	PASSED	E2
LAB-22	Deuda mínima positiva	total 120, deuda 0.01	CustomerHasDebt	Límite	PASSED	E2
LAB-23	Redondeo del descuento VIP	total 99.99 (CP12)	Descuento 15.00; cobro 84.99	Límite	PASSED	E2
LAB-24	Ventana de 24 h del propietario	propietario no admin	Denegado (el límite no se incluye)	Límite	PASSED	E6
LAB-25	Ventana de 24 h del propietario	propietario no admin	Permitido	Límite	PASSED	E6
LAB-26	No confirmar dos veces (datos CSV)	CP08: total 250	ValueError("already confirmed") (tras TDD; antes: CONFIRMED)	Límite	PASSED	E3
LAB-27	Riesgo: confirmar antes del pago	total 200, VIP	El pedido NO debería quedar CONFIRMED si el pago se rechaza	Error (riesgo documentado)	XFAIL	E5

Anexo B. Matriz completa de Eventos Nova

Tabla 11. Matriz de pruebas del PIA Eventos Nova (22 casos)

ID	Regla	Origen	Entrada	Rol	Esperado	Categoría	Estado
PIA-01	R1 Servicio adicional	BN-004, SE-004, GL-015; US-025; CA-073, CA-075	Solicitud el 10/11/2026 09:00 (10 días)	Coordinador	Registrada sin motivo; consulta a proveedores; Pendiente de aprobación del Cliente; importe = costo del proveedor (1 500.00), sin cargo propio	Éxito	PASSED
PIA-02	R1 Servicio adicional	BN-004, SE-004, GL-015; US-025; CA-083	Solicitud el 15/11/2026 23:59 (5 días naturales por fecha)	Coordinador	Procede (Registrada)	Límite	PASSED
PIA-03	R1 Servicio adicional	BN-004, SE-004, GL-015; US-025; CA-083	Solicitud el 16/11/2026 00:00 (4 días)	Coordinador	No procedente por plazo insuficiente; no se consultan proveedores	Error	PASSED
PIA-03b	R1 Servicio adicional	BN-004, SE-004; US-025; CA-083	Reintento de evaluar disponibilidad	Coordinador	Se rechaza y NO se consulta a proveedores	Error (hallazgo U5-H1)	XFAIL
PIA-04	R1 Servicio adicional	BN-004, SE-004, GL-015; US-025; CA-083	Solicitud el 17/11/2026 (3 días)	Coordinador	No procedente con el primer motivo: evento cancelado (no el de plazo)	Error	PASSED
PIA-05	R2 Anticipo	BN-009, SE-010; decisión B; US-009; CA-019	Anticipo 3 000.00 (30 % de 10 000.00)	Administración	Pago Verificado; anticipo Liquidado; contratación confirmada; evento En organización	Éxito	PASSED
PIA-06	R2 Anticipo	BN-009; decisión B; US-009; CA-019	Anticipo 2 999.99 y después 0.01	Administración	Con 2 999.99 el anticipo sigue Pendiente y no se confirma; con el complemento se liquida y se confirma	Límite	PASSED
PIA-07	R2 Anticipo	BN-009; decisión B (B-2, B-3, B-4); US-009; CA-019	Anticipo 4 000.00 (40 %)	Administración	Requisito cubierto; sin devolución; saldo del contrato sigue en 7 000.00	Éxito	PASSED
PIA-08	R2 Anticipo	BN-009, SE-010, GL-060; US-010; CA-020	Seguimiento el 06/10 23:59:59 y el 07/10 00:00	Coordinador	Antes del límite exclusivo se rechaza; a las 00:00 del 07/10 se registra	Límite	PASSED
PIA-09	R2 Anticipo	BN-009, SE-010, GL-060; US-010; CA-021	Cierre en contacto + 24 h y en + 24 h 1 s	Coordinador	En + 24 h exactas se rechaza (límite inclusivo); después: contrato Cerrado sin confirmar y evento Cerrado sin contratación (no Cancelado)	Error	PASSED
PIA-10	R3 Cambio tardío	BN-003, SE-003, GL-020; US-024; CA-061, CA-063	Cambio el 13/11/2026 18:00 (7 días); costo proveedor 800.00; tarifa ejemplo 500.00	Coordinador / Cliente / Administración	Sin cargo tardío; solo la obligación del costo del proveedor	Límite	PASSED
PIA-11	R3 Cambio tardío	BN-003, SE-005, GL-025; US-024; CA-062, CA-063	Cambio el 14/11/2026 18:00 (6 días); costo 800.00; tarifa 500.00	Coordinador / Cliente / Administración	Dos obligaciones independientes: cargo por cambio tardío 500.00 y costo del proveedor 800.00	Éxito	PASSED
PIA-12	R3 Cambio tardío	BN-003, SE-005, GL-025; US-024; CA-062	Cambio a 3 días; proveedores responden no viable	Coordinador	No procedente; no se genera ninguna obligación (no se cobra el cargo)	Error	PASSED
PIA-13	R3 Cambio tardío	BN-003, GL-060 (límite inclusivo en T); CA-039	Solicitud de cambio el 20/11/2026 18:00 (inicio)	Coordinador	Rechazo: el cambio solo procede antes del inicio; no se registra solicitud	Error	PASSED
PIA-14	R4 Pago de servicio adicional	BN-004, SE-004, GL-045; US-025; CA-080, CA-082	Pago 1 500.00 a las + 10 h	Administración / Coordinador	Servicio adicional Confirmado e incorporado al evento	Éxito	PASSED
PIA-15	R4 Pago de servicio adicional	BN-004, GL-060; US-025; CA-080, CA-082	Pago efectuado en + 24 h exactas y registrado en + 30 h	Administración / Coordinador	No se cierra (límite inclusivo); el servicio se confirma	Límite	PASSED

ID	Regla	Origen	Entrada	Rol	Esperado	Categoría	Estado
PIA-16	R4 Pago de servicio adicional	BN-004, GL-060; US-025; CA-080	Cierre en + 24 h 1 s	Coordinador	Cerrada por falta de pago; obligación Sin efecto; no se puede confirmar ni se incorpora el servicio	Error	PASSED
PIA-17	Integración	BN-009; decisión B; CA-019	Anticipo 4 000.00	Administración	Anticipo Liquidado, pago Verificado, saldo 7 000.00 y contrato confirmado leídos desde PostgreSQL	Éxito	PASSED
PIA-18	Integración	BN-004, SE-004; CA-083	Solicitud a 4 días	Coordinador	No procedente con causa Plazo insuficiente persistida	Error	PASSED
PIA-19	Integración	ADR-004 (Unidad de trabajo); —	Dos solicitudes guardadas en transacciones distintas	Sistema	La revertida no existe; la confirmada sí; el conteo aumenta en 1	Límite	PASSED
PIA-20	Integración	BN-009; CA-019	Anticipo 3 000.00, comprobante C-ANT	Administración	Primero queda Registrado y no se confirma; tras la verificación se confirma; el mock recibe (C-ANT, 3 000.00)	Error → Éxito	PASSED
PIA-21	Integración	SE-004; CA-073, CA-074	Solicitud a 10 días	Coordinador	Consulta primero a PV-1 y después la general; elige PV-9	Éxito	PASSED

Anexo C. Reproducción

```
# Laboratorio (raíz del entregable)
python -m venv .venv && source .venv/bin/activate      # Windows: .\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m ruff check src tests
python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html --cov-fail-under=80
python herramientas/generar_matrices.py                 # regenera las matrices desde
evidencias/*/junit_*.xml

# Eventos Nova: copiar pia_eventos_nova/tests_u5/ a la raíz del repositorio del equipo
python -m pip install -r requirements-dev.txt "pytest>=8,<10" "pytest-cov>=5,<8"
python -m pytest tests_u5 -v -rxX                       # PostgreSQL de pruebas opcional:
TEST_DATABASE_URL
```